

AutoDataLab++ Agent Roles Technical Explainer

AutoDataLab++: Technical Role Explainer

System Roles

In the current implementation, there are six roles in the office:

- CEO
- Chief of Staff (CoS)
- Data Analyst
- Finance
- Strategy
- HR / Communications

Technically, the contract between these roles is defined by the typed schemas in `ceo_brief_env/models.py`. The key objects are:

- `CoSAction`: what the CoS is allowed to do
- `ExpertReport`: the standard structured output from each specialist
- `Brief`: the final merged executive deliverable
- `CoSObservation`: the state the CoS sees after each step

This means the system is not just a set of chat agents. It is a typed orchestration system where specialist outputs feed a shared executive workflow.

CEO

The CEO is not currently implemented as an autonomous model in code. Instead, the CEO is represented by:

- the task instruction in the observation
- the task metadata
- the final expectation that the system should produce a decision-ready brief

Conceptually, the CEO is responsible for:

1. Defining the business problem
2. Setting the decision context
3. Establishing what a good answer should contain
4. Consuming the final brief

The CEO does not perform analysis directly. The CEO acts as the principal who defines the objective and expects a high-level synthesis.

Chief of Staff (CoS)

The Chief of Staff is the central control policy and the only trainable policy in the environment.

Its action interface supports:

- consult
- ask
- summarize
- submit
- noop

and it can route these actions to:

- analyst
- finance
- hr
- strategy

Technical Responsibilities

The CoS solves the workflow-routing problem:

- which expert to invoke
- in what sequence to invoke them
- whether to re-query an expert
- when enough information exists to summarize
- when to submit the final brief

The CoS does not compute business metrics itself. It coordinates specialists and is rewarded for good orchestration.

Data Analyst Agent

The Data Analyst is implemented in `ceo_brief_env/experts/data_analyst.py`.

Inputs

- task_name
- question
- raw_df
- focused

Core Functions Used

The Analyst relies on the data-processing substrate in `subenvs/autodatalab.analytics`:

- `clean_orders()`
- `derive_revenue()`
- `compute_kpis()`
- `compute_revenue_share()`
- `validate_schema()`
- `data_quality_score()`

Technical Responsibilities

The Analyst is the data integrity and descriptive analytics layer. It is responsible for:

1. Cleaning and normalizing raw order data

2. Measuring data quality
3. Producing core KPIs
4. Identifying leading revenue categories
5. Surfacing schema and quality risks

Output Shape

The Analyst returns an `ExpertReport` containing:

- summary text
- metrics such as total revenue, average order value, top category, and top-category revenue
- data-quality indicators
- issues and citations

System Role

The Analyst is the evidence foundation for the system. Finance, Strategy, and HR all depend on the quality of this upstream output.

Finance Agent

Finance is implemented in `ceo_brief_env/experts/finance.py`.

Inputs

- `task_name`
- `question`
- `raw_df`
- `analyst_metrics`
- `task_meta`
- `focused`

Core Functions Used

Finance uses:

- `clean_orders()`
- `monthly_revenue()`

and deterministic finance tools:

- `project_next_quarter()`
- `compute_variance()`
- `break_even()`

Technical Responsibilities

Finance is the forward-looking quantitative reasoning layer. It is responsible for:

1. Turning cleaned transaction data into monthly revenue structure
2. Forecasting next-quarter revenue
3. Comparing actual performance against plan

4. Computing break-even thresholds

Output Shape

The Finance ExpertReport contains metrics such as:

- projection_next_quarter
- confidence_band
- variance_abs
- variance_pct
- variance_flag
- break_even_units

System Role

Finance converts descriptive analytics into planning and control signals. It answers not only what happened, but what it means for near-term business execution.

Strategy Agent

Strategy is implemented in `ceo_brief_env/experts/strategy.py`.

Inputs

- task_name
- task_meta
- analyst_report
- finance_report
- focused

Technical Responsibilities

Strategy is the decision-synthesis layer. It reads the outputs of Analyst and Finance, then converts them into executive recommendations.

In the current implementation it pulls signals such as:

- top_category
- total_revenue
- projection_next_quarter
- variance_pct

and transforms them into a three-bullet operating plan.

Output Shape

The Strategy ExpertReport contains:

- a short strategic summary
- recommendation count
- recommendation bullets
- citations to upstream findings

System Role

Strategy is where the system moves from analysis to action. It is the layer that translates numbers into managerial priorities.

HR / Communications Agent

HR is implemented in `ceo_brief_env/experts/hr.py`.

Inputs

- `task_name`
- `task_meta`
- `analyst_report`
- `finance_report`
- `optional_strategy_report`
- `focused`

Core Functions Used

HR uses the communications substrate in `subenvs/email/hr_tools`:

- `build_hr_memo()`
- `score_memo()`

Technical Responsibilities

HR is the stakeholder communication layer. It is responsible for:

1. Collecting the most important highlights from upstream experts
2. Translating them into an internal memo for the intended audience
3. Scoring the memo against required terms and communication expectations

Output Shape

The HR ExpertReport contains:

- `memo_summary`
- `memo_score`
- the full memo text
- a communication-oriented output ready for inclusion in the final brief

System Role

HR makes the environment more realistic by forcing the system not only to reason correctly, but also to communicate conclusions appropriately inside an organization.

How Agent Dependencies Work

The dependencies are encoded in `ceo_brief_env/environment.py`:

- Finance can bootstrap Analyst output if Analyst has not yet been run
- Strategy depends on Analyst and Finance
- HR depends on Analyst and Finance, and optionally Strategy

This creates a partially ordered workflow rather than a flat set of independent tools.

How the Final Brief Is Built

The final brief is assembled from specialist outputs:

- metrics are merged from Analyst and Finance
- recommendations come from Strategy
- the stakeholder memo comes from HR
- the final summary concatenates upstream report summaries

The final Brief therefore acts as a composed executive artifact rather than a single-model response.

What Each Role Optimizes For

CEO

- decision usefulness
- executive clarity
- completeness of the final brief

Chief of Staff

- efficient expert routing
- low redundancy
- strong terminal brief quality

Data Analyst

- trustworthy descriptive evidence
- data quality awareness
- KPI grounding

Finance

- forecasting quality
- plan-versus-actual control
- break-even reasoning

Strategy

- actionability
- prioritization
- converting evidence into decisions

HR / Communications

- stakeholder readability
- memo quality
- organizational communication readiness

Main Technical Insight

This system should not be understood as five chatbots talking. It is better understood as a layered enterprise decision stack:

- Analyst establishes evidence
- Finance projects implications
- Strategy turns findings into action
- HR packages action for people
- CoS learns how to coordinate the workflow
- CEO defines the objective and consumes the final deliverable

That decomposition is what makes the environment useful for reinforcement learning. The hard problem is not isolated expert reasoning. The hard problem is structured multi-role orchestration under delayed reward.